

Offline Dynamic Minimum Spanning Tree

Topic.

Dynamic graph algorithms, minimum spanning tree, divide and conquer over time, roll-back DSU.

Statement.

Process offline queries of the following types:

- add an edge to the graph;
- delete an edge from the graph;
- output the current weight of the minimum spanning tree.

The goal is to answer queries faster than recomputing the minimum spanning tree from scratch after each update.

Solution.

Use divide and conquer over time.

After processing all queries, for each edge we know all time intervals during which it exists in the graph.

Build a recursion over the time segment.

For a node corresponding to a segment

$$[L, R),$$

consider all edges that intersect this segment.

Divide them into two types.

full

are the edges that exist during the whole segment

$$[L, R).$$

part

are the edges that exist only during a part of the segment

$$[L, R).$$

Maintain a DSU with rollbacks.

In each recursion node, do two things.

1. Contraction.

Run Kruskal's algorithm on the edges from *full* on the current DSU components.

All edges taken by Kruskal must belong to the MST at any time moment from

$$[L, R).$$

Add their weights to the answer immediately and unite their endpoints in the DSU.

2. Removing unnecessary edges.

Look at the edges from *part* after the current component contractions.

If an edge can no longer be taken by any Kruskal run on this level, then it will not be needed below either, so we can discard it.

After that, recursively go to the left and right child.

After returning from recursion, roll back the DSU.

In a time leaf t , all mandatory edges have already been added on the path from the root.

Using the remaining edges, finish the answer by ordinary Kruskal's algorithm and get the MST weight at time t .

If an edge from *full* is chosen by Kruskal, then it must belong to the MST for every time moment of the current segment. Therefore, it can be added immediately.

If an edge after the current contractions does not appear in any Kruskal run, then by the cycle property it cannot lie in any MST below. Therefore, the recursion keeps exactly those edges that can still affect the answer.

Complexity.

Each edge appears in

$$O(\log m)$$

nodes of the recursion over time.

At each level, it is either contracted immediately, discarded, or passed lower.

Therefore, the total running time is

$$O(m \log^2 m).$$

The memory consumption is

$$O(m \log m).$$